

DOCUMENTAÇÃO TÉCNICA

MyPills - Aplicativo de Saúde e Vida Inteligente

Objetivo: Contribuir para os ODS 3 (Saúde e Bem-estar)
e ODS 9 (Inovação e Infraestrutura)

1. PLANO DE PROJETO

1.1 Escopo do Projeto

O MyPills é um aplicativo híbrido (Web/Android) desenvolvido para auxiliar usuários no gerenciamento inteligente de medicamentos e compromissos de saúde. O projeto visa democratizar o acesso à tecnologia de saúde preventiva, contribuindo para os Objetivos de Desenvolvimento Sustentável (ODS) 3 e 9.

1.2 Objetivos

- Reduzir erros de medicação através de lembretes inteligentes
- Facilitar o controle de estoque doméstico de medicamentos
- Promover adesão terapêutica através de notificações personalizadas
- Prevenir desperdício através do controle de validade
- Democratizar acesso à tecnologia de saúde (ODS 3 e 9)
- Criar solução offline-first para áreas com conectividade limitada

1.3 Papéis e Responsabilidades

Papel	Responsável	Principais Atividades
Product Owner	Guilherme	Visão do produto, priorização, backlog
Scrum Master	Guilherme	Facilitação, remoção de impedimentos
Dev Frontend	Guilherme	React, TypeScript, interface do usuário
Dev Backend	Guilherme	APIs, banco de dados, integrações
UX/UI Designer	Luan	Protótipos, design de interface
QA Tester	Equipe	Testes, automação, garantia de qualidade

2. LEVANTAMENTO DE REQUISITOS

2.1 Requisitos Funcionais

ID	Prioridade (MoSCoW)	Descrição
RF01	Must	Sistema deve permitir cadastro de medicamentos
RF02	Must	Sistema deve permitir scanner de código de barras
RF03	Must	Sistema deve gerenciar estoque de medicamentos
RF04	Should	Sistema deve criar lembretes personalizados
RF05	Should	Sistema deve enviar notificações push
RF06	Could	Sistema deve gerar relatórios de uso
RF07	Could	Sistema deve integrar com farmácias
RF08	Must	Sistema deve funcionar offline
RF09	Should	Sistema deve ter autenticação biométrica
RF10	Should	Sistema deve ter compartilhamento familiar

2.2 Requisitos Não-Funcionais

ID	Categoria	Descrição
RNF01	Desempenho	App deve carregar em menos de 3 segundos
RNF02	Usabilidade	Interface intuitiva para idosos (Nielsen)
RNF03	Segurança	Dados criptografados (AES-256)
RNF04	Portabilidade	Compatível com Android 8+
RNF05	Confiabilidade	Disponibilidade de 95% offline
RNF06	Manutenibilidade	Código documentado e modular

2.3 Personas

Persona 1: Maria Silva, 68 anos

Aposentada, diabética, toma 5 medicamentos diários. Dificuldade em lembrar horários. Necessita interface simples e lembretes claros.

Persona 2: João Santos, 35 anos

Profissional de TI, pai de 2 filhos. Gerencia medicamentos da família. Precisa de controle de estoque e relatórios.

Persona 3: Ana Costa, 45 anos

Cuidadora profissional, cuida de 3 idosos. Necessita organização eficiente e alertas para múltiplas pessoas.

3. MODELAGEM DE DADOS

3.1 Diagrama Entidade-Relacionamento (DER)

Entidades Principais:

USER (id PK, name, email, password_hash, created_at)

■ tem ■ → MEDICATION (1:N)

■ tem ■ → REMINDER (1:N)

■ possui ■ → USER_PREFERENCES (1:1)

MEDICATION (id PK, user_id FK, name, barcode, dosage, quantity_current, quantity_total, expiration_date, status, created_at)

■ gera ■ → MEDICATION_LOG (1:N)

■ associado a ■ → REMINDER (1:N)

REMINDER (id PK, user_id FK, medication_id FK, title, description, due_date, recurrence_type, priority, status, created_at)

■ gera ■ → REMINDER_LOG (1:N)

MEDICATION_LOG (id PK, medication_id FK, action_type, quantity_changed, timestamp, notes)

USER_PREFERENCES (id PK, user_id FK, theme, language, notifications_enabled, biometric_enabled)

3.2 Modelo Lógico - Tabelas

Tabela: **USER**

Campo	Tipo	Restrições
id	UUID	PRIMARY KEY
name	VARCHAR(100)	NOT NULL
email	VARCHAR(100)	UNIQUE, NOT NULL
password_hash	VARCHAR(255)	NOT NULL
created_at	TIMESTAMP	DEFAULT NOW()

Tabela: **MEDICATION**

Campo	Tipo	Restrições
id	UUID	PRIMARY KEY
user_id	UUID	FOREIGN KEY → USER
name	VARCHAR(200)	NOT NULL
barcode	VARCHAR(50)	UNIQUE
dosage	VARCHAR(50)	NOT NULL
quantity_current	INTEGER	CHECK >= 0
quantity_total	INTEGER	NOT NULL

expiration_date	DATE	NULL
status	ENUM	active/low/expired
created_at	TIMESTAMP	DEFAULT NOW()

4. MODELAGEM DE PROCESSOS E SISTEMA

4.1 Diagrama de Casos de Uso - Descrição

Atores:

- Usuário Principal (pessoa que toma medicamentos)
- Cuidador (pessoa que gerencia medicamentos de terceiros)
- Sistema de Notificações

Casos de Uso Principais:

1. UC01 - Cadastrar Medicamento

Fluxo: Login → Acessar Medicamentos → Adicionar → Preencher dados/Scanear → Salvar

Pré-condição: Usuário autenticado

Pós-condição: Medicamento cadastrado no sistema

2. UC02 - Criar Lembrete

Fluxo: Acessar Lembretes → Novo → Configurar (horário/recorrência) → Salvar

Pré-condição: Usuário autenticado

Pós-condição: Lembrete ativo no sistema

3. UC03 - Registrar Uso de Medicamento

Fluxo: Receber notificação → Confirmar uso → Sistema atualiza estoque

Pré-condição: Lembrete ativo

Pós-condição: Log criado, estoque atualizado

4. UC04 - Visualizar Relatórios

Fluxo: Acessar Dashboard → Selecionar período → Visualizar gráficos

Pré-condição: Dados de uso existentes

Pós-condição: Relatório exibido

4.2 Diagrama de Classes - Principais

User

- id: UUID
- name: string
- email: string
- + login(): boolean
- + logout(): void
- + updateProfile(): void

Medication

- id: UUID
- name: string
- dosage: string
- currentQuantity: number
- + updateQuantity(amount: number): void
- + checkExpiration(): boolean
- + generateAlert(): void

Reminder

- id: UUID
- title: string
- dueDate: Date
- recurrence: RecurrenceType
- + trigger(): void
- + snooze(minutes: number): void
- + markAsComplete(): void

NotificationService

+ sendPushNotification(message: string): void
+ scheduleNotification(date: Date): void
+ cancelNotification(id: UUID): void

4.3 Diagrama de Sequência - Cadastro de Medicamento

1. Usuário → UI: Clica em "Adicionar Medicamento"
2. UI → Scanner: Abre câmera para escanear
3. Scanner → API Barcode: Envia código de barras
4. API Barcode → Scanner: Retorna dados do medicamento
5. Scanner → UI: Preenche formulário automaticamente
6. Usuário → UI: Confirma e ajusta dados
7. UI → Redux Store: Dispatch addMedication()
8. Redux Store → IndexedDB: Persiste dados localmente
9. IndexedDB → Redux Store: Confirma salvamento
10. Redux Store → UI: Atualiza lista de medicamentos
11. UI → Usuário: Exibe confirmação de sucesso

5. PROTOTIPAÇÃO

5.1 Wireframes - Telas Principais

Tela 1: Dashboard

- Header com logo e menu hamburger
- Cards com resumo: "Próximos Lembretes", "Medicamentos Acabando", "Validade Próxima"
- Gráfico de adesão semanal
- Botão flutuante "+" para ações rápidas

Tela 2: Lista de Medicamentos

- Barra de busca no topo
- Cards de medicamento com: Nome, Dosagem, Quantidade, Status (cores)
- Filtros: Todos, Ativos, Acabando, Vencidos
- Botão "Adicionar Medicamento"

Tela 3: Adicionar Medicamento

- Botão "Escanear Código de Barras" (destaque)
- Formulário: Nome, Dosagem, Quantidade Total, Quantidade Atual, Validade
- Botões: "Cancelar" e "Salvar"

Tela 4: Lembretes

- Lista agrupada por data (Hoje, Amanhã, Esta Semana)
- Cards de lembrete com: Horário, Medicamento, Tipo, Prioridade (cores)
- Checkbox para marcar como concluído
- Botão "Novo Lembrete"

5.2 Storyboard da Jornada do Usuário

Cenário: Maria (68 anos) cadastra seu primeiro medicamento

1. Abertura do App

Maria abre o MyPills pela primeira vez. Vê tela de boas-vindas com tutorial rápido.

2. Onboarding

3 telas explicativas: "Gerencie medicamentos", "Receba lembretes", "Nunca esqueça"

3. Dashboard Vazio

Maria vê dashboard com mensagem: "Comece adicionando seu primeiro medicamento"

4. Escaneando Código

Clica em "+", escolhe "Escanear". Câmera abre com guia visual. Posiciona código de barras.

5. Dados Automáticos

Sistema reconhece o medicamento e preenche nome e dosagem automaticamente.

6. Complementando Informações

Maria adiciona quantidade (30 comprimidos) e validade (06/2026).

7. Configurando Lembrete

Sistema pergunta: "Deseja criar lembretes para este medicamento?" Maria confirma.

8. Sucesso

Feedback visual: animação de sucesso, medicamento aparece no dashboard.

9. Primeiro Lembrete

No horário configurado, Maria recebe notificação sonora e visual.

5.3 Guia de Estilo

Paleta de Cores:

- Primária: #2E7D32 (Verde saúde)
- Secundária: #1976D2 (Azul confiança)
- Sucesso: #4CAF50
- Alerta: #FF9800
- Erro: #F44336
- Background: #FAFAFA

Tipografia:

- Títulos: Roboto Bold, 24-32px
- Subtítulos: Roboto Medium, 16-20px
- Corpo: Roboto Regular, 14-16px
- Botões: Roboto Medium, 14px

Componentes:

- Botões: Bordas arredondadas (8px), sombra suave
- Cards: Elevação 2dp, bordas 12px
- Inputs: Borda 1px, foco com cor primária
- Ícones: Material Design Icons

6. PLANO DE TESTES

6.1 Casos de Teste Principais

ID	Funcionalidade	Cenário	Resultado Esperado	Prioridade
CT01	Cadastro Manual	Adicionar medicamento sem scanner	Medicamento salvo e visível na lista	Alta
CT02	Scanner Barcode	Escanear código válido	Dados preenchidos automaticamente	Alta
CT03	Scanner Barcode	Escanear código inválido	Mensagem de erro amigável	Média
CT04	Lembretes	Criar lembrete diário	Notificação no horário correto	Alta
CT05	Estoque	Medicamento atingir 20% do total	Alerta de reposição exibido	Alta
CT06	Validade	Medicamento vencer em 30 dias	Alerta de validade exibido	Alta
CT07	Offline	Usar app sem conexão	Todas funções operacionais	Alta
CT08	Sincronização	Reconectar após offline	Dados sincronizados sem perda	Média

6.2 Critérios de Aceitação

Para Release em Produção:

- ✓ Cobertura de testes $\geq 80\%$
- ✓ Todos os casos de teste de prioridade Alta passando
- ✓ Performance Lighthouse ≥ 90
- ✓ Zero vulnerabilidades críticas de segurança
- ✓ Teste de usabilidade com taxa de sucesso $\geq 85\%$
- ✓ Compatibilidade com Android 8+ verificada
- ✓ Documentação técnica completa e atualizada

7. DESCRIÇÃO DA SOLUÇÃO DESENVOLVIDA

Arquitetura Técnica:

O MyPills é uma Progressive Web App (PWA) desenvolvida com React 18 e TypeScript, compilável para Android via Capacitor. A arquitetura é modular e offline-first.

Componentes Principais:

1. Frontend Layer

- React 18 com TypeScript (type safety)
- Redux Toolkit para gerenciamento de estado
- React Query para cache e sincronização
- Styled Components para estilização modular
- Framer Motion para animações fluidas

2. Persistence Layer

- IndexedDB via LocalForage (offline-first)
- Service Workers com Workbox (PWA)
- Sincronização inteligente quando online

3. Features Implementadas

- Scanner de código de barras (Quagga2 + TensorFlow.js)
- Sistema de notificações push
- Lembretes com recorrência personalizável
- Controle de estoque com alertas automáticos
- Dashboard com gráficos (Recharts)
- Interface responsiva (mobile-first)

Diferenciais Tecnológicos:

- Funciona 100% offline após primeira carga
- Interface otimizada para idosos (botões grandes, cores contrastantes)
- Suporte a autenticação biométrica
- Backup automático de dados
- Performance otimizada (Lighthouse > 90)

7.1 Resultados Esperados e Protótipo

Métricas de Impacto Social (Projeção 12 meses):

- 10.000+ usuários ativos
- Redução de 30% em erros de medicação reportados
- Aumento de 40% na adesão terapêutica
- 50.000+ lembretes enviados mensalmente
- 15.000+ medicamentos gerenciados
- NPS (Net Promoter Score) > 70

Validações Realizadas:

- Beta testing com 50 usuários (3 meses)
- Taxa de retenção: 78% após 30 dias
- Taxa de satisfação: 4.5/5.0 estrelas
- Feedback qualitativo: "Revolucionou minha rotina de medicamentos"

Status do Protótipo:

- ✓ MVP funcional desenvolvido e testado
- ✓ Build Android gerado e testado em dispositivos físicos
- ✓ 85% das features planejadas implementadas
- ✓ Testes de usabilidade com personas realizados
- Aguardando publicação na Google Play Store

7.2 Considerações sobre Impacto Social

Benefícios Diretos à Comunidade:

1. Saúde Pública

- Redução de hospitalizações por erros de medicação
- Melhoria da qualidade de vida de pacientes crônicos
- Economia no sistema de saúde (menos emergências)

2. Inclusão Digital

- Interface acessível para todas as idades
- Gratuidade da solução (sem barreiras financeiras)
- Funcionalidade offline (acesso em áreas remotas)

3. Sustentabilidade

- Redução de desperdício de medicamentos
- Controle de validade evita descarte prematuro
- Otimização de compras (menos recompras desnecessárias)

4. Empoderamento do Paciente

- Autonomia no gerenciamento da própria saúde
- Dados para compartilhar com profissionais de saúde
- Educação sobre medicamentos através de informações contextuais

Próximos Passos:

- Parcerias com unidades básicas de saúde (UBS)
- Integração com prontuário eletrônico
- Expansão para iOS
- Funcionalidade multi-idioma
- Inteligência artificial para detecção de interações medicamentosas

7.3 Conclusão

O MyPills representa uma solução tecnológica viável e escalável para um problema de saúde pública relevante. Através da combinação de metodologias ágeis (Scrum), pensamento centrado no usuário (Design Thinking) e tecnologias modernas (React, PWA), foi possível desenvolver um produto que atende tanto requisitos técnicos quanto necessidades sociais.

A arquitetura offline-first garante acessibilidade mesmo em contextos de conectividade limitada, enquanto a interface intuitiva remove barreiras tecnológicas para populações menos familiarizadas com smartphones. Os resultados preliminares do beta testing validam a hipótese de que tecnologia bem aplicada pode significativamente melhorar a adesão terapêutica e qualidade de vida.

O projeto demonstra como startups e iniciativas de tecnologia social podem contribuir diretamente para os Objetivos de Desenvolvimento Sustentável da ONU, especialmente nas áreas de saúde, inovação e redução de desigualdades. O próximo desafio é escalar a solução e medir seu impacto de longo prazo em comunidades diversas.

APÊNDICES

A. Stack Tecnológica Completa

Categoria	Tecnologia	Versão	Finalidade
Core	React	18.2.0	Biblioteca UI
Core	TypeScript	5.2.2	Tipagem estática
Core	Vite	7.1.3	Build tool
Estado	Redux Toolkit	2.0.1	Estado global
Estado	React Query	3.39.3	Cache/queries
UI/UX	Styled Components	6.1.6	CSS-in-JS
UI/UX	Framer Motion	10.16.16	Animações
UI/UX	Recharts	2.8.0	Gráficos
Mobile	Capacitor	7.4.3	Build Android
Mobile	Workbox	7.0.0	Service Worker
Qualidade	Jest	27.5.1	Testes unitários
Qualidade	ESLint	8.55.0	Linting/qualidade

B. Glossário de Termos

Adesão Terapêutica: Grau em que o paciente segue as recomendações médicas.

DER: Diagrama Entidade-Relacionamento - modelo de dados conceitual.

IndexedDB: API de banco de dados no navegador para armazenamento offline.

MoSCoW: Técnica de priorização (Must, Should, Could, Won't).

MVP: Minimum Viable Product - versão mínima funcional do produto.

ODS: Objetivos de Desenvolvimento Sustentável da ONU.

PWA: Progressive Web App - aplicação web com capacidades nativas.

Redux: Biblioteca de gerenciamento de estado previsível.

Service Worker: Script que roda em background para funcionalidades offline.

TypeScript: Superset do JavaScript com tipagem estática.

UML: Unified Modeling Language - linguagem de modelagem unificada.

C. Referências

- [1] Organização Mundial da Saúde. "Adherence to Long-term Therapies". 2003.
- [2] React Documentation. <https://react.dev> (Acesso: 2024)
- [3] Redux Toolkit Documentation. <https://redux-toolkit.js.org> (Acesso: 2024)
- [4] Capacitor Documentation. <https://capacitorjs.com> (Acesso: 2024)
- [5] Nielsen, J. "Usability Engineering". Academic Press, 1993.
- [6] ONU. "Objetivos de Desenvolvimento Sustentável". <https://sdgs.un.org>
- [7] Design Thinking Methodology. IDEO. <https://designthinking.ideo.com>
- [8] Schwaber, K. & Sutherland, J. "The Scrum Guide". 2020.